INVENTORY MANAGEMENT SYSTEM

Instructor: Dr. Hamad Naeem

GROUP INFORMATION

- Abdullah Masoud Alsubaie. - 222421103.
- Hamza eyad Mohammed – 222453516.

Section – 13.

INTRODUCTION

The Inventory Management System provides a practical solution for businesses to efficiently organize and control their product inventory. This console-based application leverages object-oriented programming principles to manage product information using linked lists. Users can easily perform key operations, including adding, removing, updating, and displaying products, making it a user-friendly tool for small to medium-sized businesses looking for a straightforward inventory management solution.

Establishing Linked list

01

In order to establish a linked list in c++, you have to follow these steps:

1. Define a struct or class for your linked list node. In our project we are going to use class:

```
class Node
{
private:
    string name;
    int productID;
    int quantity;
    double price;
    Node* next;
```

Here, we defined the class for our linked list node that includes the product's attributes which is : name , ID , quantity and price.

2. Create a pointer.

```
Node* next;
```

This satisfies the purpose of pointing to the next node.

In order to establish a linked list in c++, you have to follow these steps:

3. Create a constructor in the 'Node' class to initialize the product data.

```
public:
    Node(string name, int productID, int quantity, double price)
    {
        this->name = name;
        this->productID = productID;
        this->quantity = quantity;
        this->price = price;
        next = nullptr;
    }
```

The constructor plays an important role as it ensures that when a new 'Node' object is created, it has initial values for its data members.

Note: Without a constructor, these values might be uninitialized, leading to unexpected behavior.

In order to establish a linked list in c++, you have to follow these steps:

4. Set and get methods

```
void setName(string newName) { name = newName; }
string getName() { return name; }

void setProductID(int newProductID) { productID = newProductID; }
int getProductID() { return productID; }

void setQuantity(int newQuantity) { quantity = newQuantity; }
int getQuantity() { return quantity; }

void setPrice(double newPrice) { price = newPrice; }
double getPrice() { return price; }

void setNext(Node* newNext) { next = newNext; }
Node* getNext() { return next; }
```

CLASS INVENTORY

Now that we are done making the **'Node'** class, we make a class **'Inventory'** that include all the needed functions in this project.

```
class Inventory
{
private:
    Node* head;

public:
    Inventory() : head(nullptr) {}
```

- A private pointer that points to the head of the linked list.

- A constructor to initialize the value (null) to the pointer head.

MAIN MENU

MAIN MENU

When the user runs the program, the program asks the user to press Enter to continue.

```
Welcome To Inventory Manegment System

Press Enter to continue.
```

After pressing Enter, the program will display the main menu that includes a few options to choose from.

```
------------------------

        Menu:

1. Add a product
2. Remove a product
3. Update a product
4. Display inventory
5. Exit

Enter your choice:
```

```
int main()
{
    Inventory inventory;
    int choice;


    system("cls");
    cout << "\n\n\n\n\n\t\t\t\t Welcome To Inventory Manegment System\n\n\t\t\t\t\tPress Enter to continue.";
    cin.get();

    do
    {
        system("cls");
        cout << "------------------------" << endl;
        cout << "\n          Menu:" << endl;
        cout << "\n1. Add a product" << endl;
        cout << "2. Remove a product" << endl;
        cout << "3. Update a product" << endl;
        cout << "4. Display inventory" << endl;
        cout << "5. Exit" << endl;
        cout << "\nEnter your choice: ";
        cin >> choice;
        cout << endl;
        system("cls");
```

In case of choosing 1 (Add a product)...

ADD A PRODUCT

ADD A PRODUCT

```
case 1:
{
    int num;
    string name;
    int productID, quantity;
    double price;
    cout << "\t--add a product--\n\n";
    cout << "How many products would you like to add? ";
    cin >> num;
    for (int i = 0; i < num; i++)
    {
        cout << "\nEnter product '" << i+1 << "' name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter product ID: ";
        cin >> productID;
        while (!inventory.isNewId(productID))
        {
            cout << "Product ID already exists. Please enter a new ID: ";
            cin >> productID;
        }
        cout << "Enter quantity: ";
        cin >> quantity;
        cout << "Enter price: ";
        cin >> price;
        inventory.addProduct(name, productID, quantity, price);
    }
    cout << "\t\t\t\t \nPress Enter to continue  \n";
    cin.ignore();
    cin.get();
    system("cls");

    break;
}
```

First, The program will ask the user how many product he wants to add.

After entering the number of products, the program will ask the user to enter each product's information individually (name , ID , quantity and price).

*The given ID will be sent to a function to check if the ID entered already exists or not. If the ID already exists, the program will keep asking the user for a valid ID until one is entered.

Lastly, The program will display a message informing the user that the products have been added successfully,

After all required information is given, it will be sent to 'addProduct' function

ADD A PRODUCT

(inside the function)

```
void addProduct(string name, int productID, int quantity, double price)
{
    Node* newNode = new Node(name, productID, quantity, price);

    if (head == nullptr)
    {
        head = newNode;
    }
    else
    {
        Node* current = head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
    cout << "Product added to inventory." << endl << endl;
}
```

First, The program will create a new Node to store all the information.

Second, the program will check the list, if the head pointer equals null, it means that the list is empty and the program will make the new Node be the first node in the list.

Otherwise, the program will make the new Node be the last node by sweeping through all Nodes in the list until the next Node of the current Node is equal to null

```
Enter your choice: 1

Enter product name: Book
Enter product ID: 101
Enter quantity: 1
Enter price: 99.99
Product added to inventory.

------------------------
```

Lastly, the program will print a message to inform the user that the product has be successfully added to the inventory.

ADD A PRODUCT

```
        Menu:

1. Add a product
2. Remove a product
3. Update a product
4. Display inventory
5. Exit

Enter your choice: 1

sh: cls: command not found
    --add a product--

How many products would you like to add? 2

Enter product '1' name: A song of ice and fire
Enter product ID: 100
Enter quantity: 1
Enter price: 499.99
Product added to inventory.


Enter product '2' name: Endless story
Enter product ID: 101
Enter quantity: 1
Enter price: 199.99
Product added to inventory.


Press Enter to continue
```

REMOVE A PRODUCT

REMOVE A PRODUCT

Incase of choosing option 2 (Remove a product)...

```
case 2:
{
    int productID;
    cout << "\t--remove a product--\n\n";
    cout << "Enter product ID to remove: ";
    cin >> productID;
    if (inventory.isFound(productID))
    {
        inventory.removeProduct(productID);
    }
    else
    {
        cout << "\nProduct not found!" << endl;
    }
    cout << "\t\t\t\t \nPress Enter to continue  \n";
    cin.ignore();
    cin.get();
    system("cls");
    break;
}
case 3:
```

First, the program will ask the user to enter the product's ID that the user wants to remove.

After entering the ID, it will be sent to a function to check if the product exists or not.

If it exists, the program will send the ID to a function that deletes the product.

Otherwise, the program will display a message saying that the product was not found.

```
        Menu:

1. Add a product
2. Remove a product
3. Update a product
4. Display inventory
5. Exit

Enter your choice: 2

sh: cls: command not found
    --remove a product--

Enter product ID to remove: 101
Product has been removed from inventory.


Press Enter to continue
```

REMOVE A PRODUCT

(inside the function)

```
void removeProduct(int productID)
{
    if (head == nullptr)
    {
        cout << "Inventory is empty." << endl << endl;
        return;
    }


    if (head->productID == productID)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "\nProduct has been removed from inventory." << endl << endl;
        return;
    }

    Node* current = head;
    while (current->next != nullptr)
    {
        if (current->next->productID == productID)
        {
            Node* temp = current->next;
            current->next = temp->next;
            delete temp;
            cout << "Product has been removed from inventory." << endl << endl;
            return;
        }
        current = current->next;
    }

    cout << "Product was not found in inventory." << endl << endl;

}
```

First, the program will check the list, if the head pointer equals null, it means that the list is empty and the program will print a message stating that the list is empty.

Then, if the product is at the head of the list,

The program will create a temporary Node that points to the head, then the head Node will point to the next Node and temp will be deleted.

A message will appear stating the the product has been removed successfully.

REMOVE A PRODUCT

(inside the function)

```
void removeProduct(int productID)
{
    if (head == nullptr)
    {
        cout << "Inventory is empty." << endl << endl;
        return;
    }


    if (head->productID == productID)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "\nProduct has been removed from inventory." << endl << endl;
        return;
    }

    Node* current = head;
    while (current->next != nullptr)
    {
        if (current->next->productID == productID)
        {
            Node* temp = current->next;
            current->next = temp->next;
            delete temp;
            cout << "Product has been removed from inventory." << endl << endl;
            return;
        }
        current = current->next;
    }

    cout << "Product was not found in inventory." << endl << endl;

}
```

If the product is somewhere in the list,
The program will create a current Node that points at the head.

While the next Node is NOT equal to null,

If the ID at the next Node is equal to the give ID,
The program will create a temporary node that points at the next node, then it will make the next node equal the value of the temporary node and the temporary node will be deleted after.

A message will appear stating that the product has been removed successfully and the current node will point at the next Node.

UPDATE A PRODUCT

UPDATE A PRODUCT

Incase of choosing option 3 (Update a product)...

```
case 3:
{
    int productID, newQuantity;
    double newPrice;
    cout << "\t--ubdate a product--\n\n";
    cout << "Enter product ID to update: ";
    cin >> productID;
    if (inventory.isFound(productID))
    {
        cout << "\nEnter new quantity: ";
        cin >> newQuantity;
        cout << "Enter new price: ";
        cin >> newPrice;
        inventory.updateProduct(productID, newQuantity, newPrice);
        cout << "\t\t\t\t \nPress Enter to continue  \n";
        cin.ignore();
        cin.get();
        system("cls");
    }
    else
    {
        cout << "\nProduct not found!" << endl;
        cout << "\t\t\t\t \nPress Enter to continue  \n";
        cin.ignore();
        cin.get();
        system("cls");
    }
    break;
}
```

First, the program will ask the user to enter the product's ID that the user wants to update.

After entering the ID, it will be sent to a function to check if the product exists or not.

If it exists, the program will ask the user to enter the new quantity and price of the product.

After entering the new information, it will be sent to a function to update it.

Otherwise, the program will display a message saying that the product was not found.

```
Enter product ID to update: 100

Enter new quantity: 3
Enter new price: 899.99
Product has been updated successfully.


Press Enter to continue
```

UPDATE A PRODUCT

(inside the function)

```
void updateProduct(int productID, int newQuantity, double newPrice)
{
    Node* current = head;
    while (current != nullptr)
    {
        if (current->productID == productID)
        {
            current->quantity = newQuantity;
            current->price = newPrice;
            cout << "Product has been updated successfully." << endl << endl;
            return;
        }
        current = current->next;
    }
    cout << "Product was not found in inventory." << endl << endl;
}
```

First, the program will create a Node pointer called current that points to the head

While the current pointer is NOT equal to null pointer...

If the current ID equals the given ID, the program will make the current quantity and price equals the new quantity and price.

Then, the program will display a message states that the product has been updated

If the current pointer is indeed equal to null pointer it means that the product wasn't found.

DISPLAY INVENTORY

DISPLAY INVENTORY

Incase of choosing option 4 (Display inventory)...

```
case 4:
{
    inventory.displayInventory();
    cout << "\t\t\t\t \nPress Enter to continue  \n";
    cin.ignore();
    cin.get();
    system("cls");
    break;
}
```

```
                *Inventory*

Name    |   ID  |   Quantity    |   Price

Book 1  |   100 |   1       |   199.99
Book 2  |   101 |   5       |   299.99
Book 3  |   102 |   18      |   10
Book 4  |   103 |   39      |   5.99

Press Enter to continue
```

The display inventory function will be immediately called.

DISPLAY INVENTORY

(inside the function)

```
    void displayInventory()
    {
        if (head == nullptr)
        {
            cout << "Inventory is empty." << endl << endl;
            return;
        }

        cout << "\t\t\t*Inventory*" << endl << endl;

        Node* current = head;
        cout << "Name\t|\tID\t|\tQuantity\t|\tPrice\n\n";
        while (current != nullptr)
        {

            cout << current->name << "\t|\t" << current->productID << "\t|\t" << current->quantity << "\t\t|\t" << current->price << endl;
            current = current->next;
        }
    }

};
```

First, if the head pointer equals null, it means that the list is empty and the program will print a message stating that the inventory is empty.

Otherwise, the program will create a current pointer that equals the head pointer to use later.

While the current ptr is NOT equal to null ptr, the program will keep printing all the products information until the current pointer is indeed equal null pointer.

EXIT

EXIT

Incase of choosing option 5 (Exit)...

```
case 5:
{
    cout << "Exiting program." << endl;
    cout << "\t\t\t\t \nPress Enter to continue  \n";
    cin.ignore();
    cin.get();
    system("cls");
    break;
}
```

The program will print a message to inform the user that the program is exiting.

Keep in mind that entering a number that isn't from 1 - 5 will not be accepted and the program will keep asking to enter a valid number.

```
default:
{
    cout << "Invalid choice. Please try again." << endl;
    cout << "\t\t\t\t \nPress Enter to continue  \n";
    cin.ignore();
    cin.get();
    system("cls");
    break;
}
```

Thank you for listening.